3. Encapsulation

**

What

- binds the data and methods in a class ()
- Something like a capsule
  - Single ball doesn't make any sense but
  - Capsule as a whole makes
- provides a secure layer
- Hides the internal implementation
- Exposes only necessary information
- Also call **data hiding**
- the goal is to implement a class in such a way that prevents unauthorized access to the data of the object

`Private` , `Public` , `Protected`

Public

- One can access anywhere he want to access
  - Outside the class
  - Within
  - While inherited

Private

- One can access the attributes/methods only inside the class
  - Accessing outside will result in an error `compile-time error`
  - Can't be accessed in the derived class

Protected

- It behaves the same as private when trying to access outside the class
- but changes when
  - tried to be accessed in the derived class
  - No error

Summary

	`Public`	`Private`	`Protected`
Inside the class	Works ✅	Works ✅	Works ✅
Outside the class	Works ✅	Error ❌ (Compile-time error)	Error ❌ (Compile-time error)
In the derived Class	Works ✅	Error ❌ (Compile-time error)	Works ✅

Getters and Setters

- These are special functions used to edit/read the data of the `private` || `protected` members

C++ **Using getters and setters**

```cpp
class Student{
    private:
    int id,a,b,c;

    public:
    void setId(int id){
        this->id = id;
    }
    int getId(){
        return this->id;
    }
};
```

- now carefully see
  - we have added another layer of **security**